Failure Analysis of Lightweight LLM Code Agents for Software Repair

Rishabh Singh[a,*]

[a]*Independent Researcher, Seattle, Washington, United States*

Abstract

Language-model code agents increasingly produce patches, explanations, and test-related changes for software-maintenance tasks, yet final benchmark scores often hide why generated artifacts fail. This paper reports an empirical failure analysis of lightweight code-agent workflows. A calibration layer first evaluates six Python functions with three summarization systems and a small defect heuristic. Adding ROUGE-L and METEOR to BLEU-1 shows that the rule baseline wins on the curated summary set, while the language-model baseline produces plausible paraphrases with lower surface overlap. The bug detector succeeds on two handcrafted examples but detects none of thirty BugsInPy-derived Tests4Py bug records, giving zero recall and 0.50 accuracy on the external stress set. The repository layer first evaluates direct prompting and agentless staged prompting on 50 SWE-bench Lite instances using OpenAI `gpt-4o-mini`, `Qwen2.5-Coder-7B-Instruct`, and `Qwen2.5-Coder-32B-Instruct-AWQ`. For `gpt-4o-mini`, staging resolves 1 task versus 0 for direct prompting. For Qwen 7B, both modes resolve the same single task. For Qwen 32B AWQ, staging resolves 4 tasks versus 1 for direct prompting, with McNemar's exact test giving $p = 0.25$. A larger repaired-and-evaluated extension then runs the same infrastructure on 300 `gpt-4o-mini` instances, 300 Qwen 32B AWQ instances, and a deterministic 200-instance Qwen 7B subset. The larger runs resolve 3/300 versus 1/300 for `gpt-4o-mini`, 8/300 versus 6/300 for Qwen 32B AWQ, and 2/200 versus 2/200 for Qwen 7B, with paired exact tests remaining descriptive rather than significant. Across the runs, staged prompting modestly increases completed

*Corresponding author.

Email address: `rishabhsingh@gwu.edu` (Rishabh Singh)

harness executions more consistently than it improves final resolution. The dominant bottleneck is repository-grounded patch validity: non-empty diffs often remain malformed, stale, mislocated, reversed, or semantically insufficient after application.

Keywords: language models, software engineering, code agents, bug detection, empirical evaluation, repository benchmarks

---

1. Introduction

Software-engineering tools based on language models are now used for explanations, completions, debugging, tests, and repository search. The evidence for their value, however, is not as simple as the public benchmark tables make it appear. Function-level datasets such as HumanEval, MBPP, and DS-1000 made executable code generation measurable [1, 2, 3]. They remain useful, but they leave out much of day-to-day maintenance work: finding the relevant files, understanding local conventions, editing several locations, and making sure unrelated behavior is not broken.

Repository-level work pushes closer to that setting. RepoBench, SWE-bench, SWE-agent, Agentless, SWE-bench Live, FEA-Bench, and SecRepoBench all move the discussion away from isolated snippets and toward repository context, issue repair, feature work, tool use, and security-sensitive validation [5, 6, 7, 8, 9, 10, 11]. At the same time, productivity studies do not all point in the same direction. Some controlled or field settings show faster task completion with assistance, while another study reports slower completion for experienced maintainers working on mature open-source projects [12, 13, 14, 15]. A useful evaluation should therefore ask more than whether a patch passed one benchmark. It should also ask what context the system received, what process it followed, what it cost, what it failed to validate, and whether a developer would trust the result.

The pilot reported here starts from a narrower problem. An earlier version of the study used BLEU-1 by itself for summary scoring and checked defect detection on only two deliberately inserted bugs. Those choices made the results too easy to read as stronger than they were. This revision keeps the pilot scope visible but repairs the evaluation design in three ways. It adds ROUGE-L and METEOR beside BLEU-1, reports two non-model summary baselines plus one zero-shot language-model baseline, and tests the detector against thirty BugsInPy-derived Tests4Py bug records paired with fixed

records.

The second layer moves the same cautionary question into repository repair. If a rule-based or lightweight prompting method looks useful on controlled inputs, the next question is whether its outputs remain usable after repository checkout, patch application, Docker evaluation, and test execution. The SWE-bench Lite runs therefore compare direct prompting with an agentless staged protocol using gpt-4o-mini, Qwen2.5-Coder-7B-Instruct, and Qwen2.5-Coder-32B-Instruct-AWQ. The two layers belong in one study because both test the same practical risk for LLM-assisted software-maintenance workflows: surface-level generated text can look plausible while failing once it is externally validated.

The paper makes five contributions for empirical software-engineering evaluation. First, it shows how a small curated pilot can overstate utility when metrics and defect examples are too narrow. Second, it reports paired SWE-bench Lite executions for direct and staged prompting across one proprietary lightweight model and two open-weight code-model deployments. Third, it provides a failure taxonomy that distinguishes unresolved tests from malformed diffs, hunk/context mismatches, wrong paths, duplicated or reversed changes, and empty outputs. Fourth, it reports cost and executability measures alongside resolve rate, emphasizing outcomes that matter before code-agent outputs can be trusted in maintenance workflows. Fifth, it extends the repository experiment to larger repaired-and-evaluated SWE-bench Lite runs, illustrating why prediction generation, patch repair, Docker execution, and statistical comparison must be treated as distinct stages. The paper is organized as follows: Introduction, Method, Results and Discussion, and Conclusion.

1.1. Related work and research gap

1.1.1. Function-level benchmarks

Early code-model benchmarks focused on compact programming problems. HumanEval tied generation to executable tests for docstring-to-code tasks [1]. MBPP broadened short natural-language programming problems [2]. DS-1000 moved into library-heavy Python data-science tasks and used perturbations to reduce memorization risk [3]. EvalPlus later showed that weak tests can overstate correctness and even alter model rankings [4]. These datasets are still valuable for calibration and harness debugging. They are not, by themselves, a convincing substitute for repository maintenance, be-

cause they do not test navigation, multi-file editing, maintainability, or regression preservation.

1.1.2. Repository-level benchmarks

A second family of benchmarks treats the repository as part of the task. RepoBench focuses on repository-level completion and retrieval behavior [5]. SWE-bench frames maintenance as patch verification: the system receives an issue and a repository snapshot, proposes an edit, and is judged by its evaluation tests [6]. SWE-bench Live was introduced to reduce staleness by using newer tasks and reproducible execution settings [9]. FEA-Bench shifts attention toward feature implementation [10], while SecRepoBench adds the question of whether generated code is secure as well as functional [11]. These benchmarks are harder to run than function tests, but they match more of the engineering situation that practitioners care about.

1.1.3. Agents and interfaces

A model is not the same thing as a coding system. SWE-agent studies what happens when a language model interacts with a software interface that offers search, edit, and execution operations [7]. Agentless makes the opposite point in a useful way: a staged non-agent procedure can still be competitive without giving the system broad autonomous control [8]. For that reason, evaluations should report the interface condition explicitly. Direct prompting, staged prompting, retrieval, and tool-enabled execution are different interventions, even when the same underlying model is used.

1.1.4. Defect finding and validation

Bug detection is especially sensitive to unsupported claims. A language model may produce a convincing bug explanation while being wrong about the actual program behavior. Wang et al. study this problem by requiring candidate data-flow evidence and screening it with program-property checks; their reported results show that validation structure matters for bug-reasoning tasks [16]. The implication for this pilot is straightforward: a bug detector should not be trusted because it finds two handcrafted examples. It should be pushed against external records, and its failures should be reported rather than hidden.

1.1.5. Gap analysis

No single benchmark family covers the full engineering story. Function-level tests are executable and cheap, but narrow. Repository benchmarks

are closer to practice, but they are expensive and may become stale or contaminated. Agent studies show that scaffolding affects outcomes, but they can also mix model ability with product-interface behavior. Security benchmarks remind us that ordinary correctness is not enough. The contribution of this paper is deliberately modest: it reports a concrete pilot with stronger metrics plus outside bug records, then adds three initial SWE-bench Lite backend configurations to test whether the staged protocol improves patch executability and task resolution under a real Docker harness. In both parts of the study, the central object of analysis is not a headline score but the gap between apparently successful generated text and artifacts that remain useful after external validation.

2. Method

2.1. Research questions

The executed study answers six questions:

- **RQ1:** How do the summarization conclusions change when BLEU-1 is reported with ROUGE-L and METEOR?
- **RQ2:** How does the deterministic rule-based summarizer compare with a name-only baseline and a zero-shot language-model baseline?
- **RQ3:** Does the handcrafted bug-detection heuristic transfer beyond the two original pilot bugs?
- **RQ4:** How do the pilot and repository results jointly inform language-model evaluation for software maintenance?
- **RQ5:** In 50-instance SWE-bench Lite runs with gpt-4o-mini, Qwen2.5-Coder-7B-Instruct, and Qwen2.5-Coder-32B-Instruct-AWQ, does agentless staged prompting improve task resolution or patch executability over direct prompting?
- **RQ6:** What changes when the same direct and staged infrastructure is extended to larger repaired-and-evaluated SWE-bench Lite runs?

The following questions remain protocol-design questions for a larger follow-up, not empirical claims of the present paper:

- **PDQ1:** Does broader tool use add benefit beyond retrieval and staged reasoning?
- **PDQ2:** Do live or contamination-aware benchmarks change rankings relative to static calibration benchmarks?
- **PDQ3:** Which process variables, including localization quality, retrieval precision, and validation-loop success, predict final task success?

- **PDQ4:** Does security-aware evaluation change conclusions drawn from ordinary correctness alone?

2.2. Summarization dataset

The summarization pilot uses six Python functions: factorial computation, mean calculation, primality testing, sorting, Fibonacci computation, and maximum-element selection. Each function has one reference summary written by the author. The set is intentionally small. Its purpose is to check the evaluation pipeline and to expose how easily a narrow task can reward a hand-tuned method.

Three systems are compared. The *name-only baseline* converts the function name into whitespace-separated tokens. The *rule-based baseline* maps the known recognized names into short descriptions. The *zero-shot language-model baseline* was generated in ChatGPT with the model picker set to *Thinking*; OpenAI's ChatGPT documentation identifies that setting as GPT-5.5 Thinking [25]. The run date was 2026-05-07, and the prompt was: "Provide one short English summary of the following Python function." Because this was not executed through a pinned public API checkpoint with provider logs, this result is used as an illustrative comparison inside the pilot, not as a vendor benchmark or leaderboard result.

2.3. Summarization metrics

BLEU-1 is reported with a brevity penalty. Without that penalty, a very short output can look better than it should. ROUGE-L is added because longest-common-subsequence overlap captures some sequence-level similarity that unigram precision misses [17]. METEOR is added because it combines precision and recall and penalizes fragmented alignments [18]. In the local artifact, METEOR uses exact matches and Porter-stem matches only. Synonym matching is disabled because WordNet resources were not available in the offline environment. That limitation is kept visible so that the metric values can be reproduced.

2.4. Bug-detection datasets

The original pilot set contains six small Python functions. Two are defective by construction: the arithmetic-mean function returns `None` for an empty list, and the Fibonacci function contains an off-by-one error. The heuristic checks for a small set of known patterns and edge-case failures. This design was useful for a smoke test, but it was not a meaningful generalization test.

To add external pressure, the study includes a BugsInPy-derived Tests4Py stress set. Tests4Py is a benchmark for system testing and debugging derived from BugsInPy, and it provides real-world Python bug subjects [19]. Thirty registered bug records were sampled from local Tests4Py package metadata across projects such as Ansible, Black, FastAPI, Keras, Luigi, Matplotlib, Pandas, Scrapy, TheFuck, Tornado, tqdm, and youtube-dl. Each buggy record was paired with its fixed-version record, giving sixty external stress-test instances. No project-specific rules were added to the detector. That choice is intentional: the experiment asks whether the original toy heuristic transfers without being repaired after seeing the external data.

The external stress test is still not a full source-level bug-localization benchmark. The artifact converts each Tests4Py record into a normalized textual record containing the project name, bug identifier, bug-or-fixed label, registered test metadata, and locally available descriptor fields. The same toy-pattern rules are then applied to that textual record. A record is labeled buggy only if one of the original patterns appears. The zero-recall result is therefore interpreted as a metadata-level transfer failure, not as a claim about executing or repairing full repository states.

2.5. Comparison workflow

Figure 1 shows the revised pilot workflow. The two empirical tracks are kept separate: summarization goes through the three text metrics, while bug detection goes through the Tests4Py-derived stress set and the validity analysis.

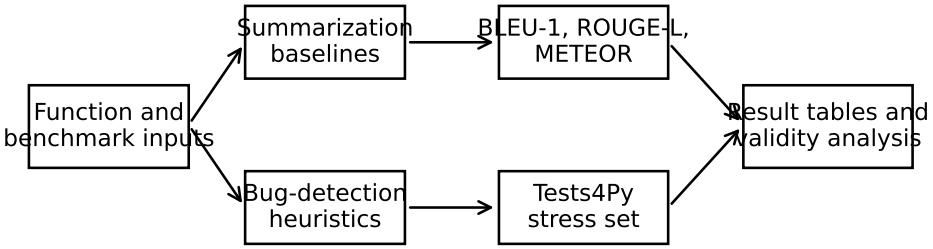


Figure 1: Revised pilot evaluation workflow.

2.6. Repository-level protocol and executed slice

The pilot results motivate a broader repository-level protocol. The current paper executes three lightweight backend configurations from that protocol on SWE-bench Lite, first as a 50-instance diagnostic slice and then as a larger repaired-and-evaluated extension. Tool-augmented agents, human review, and live or security-focused benchmark tiers remain design elements for a fuller follow-up.

2.6.1. Benchmark tiers

The broader study should use a tiered benchmark design. The calibration tier should use HumanEval, MBPP, and DS-1000 to debug prompts and evaluation code. The repository runs in this paper use SWE-bench Lite because it provides a stable Docker-based issue-repair setting suitable for controlled execution. A later version should move from SWE-bench Lite to SWE-bench Live, because live or recent tasks reduce staleness risk. Secondary validation with FEA-Bench and SecRepoBench should be added only after the primary repair experiment runs cleanly; feature implementation and secure completion add separate validity demands.

2.6.2. Model and interface separation

The follow-up should separate base models from product surfaces. A base model is the inference target. A coding assistant is an interface that may add retrieval, sandboxing, repository instructions, tool permissions, and validation behavior. If those layers are merged in the analysis, the result may be impossible to interpret: an apparent model improvement may actually come from the surrounding interface.

2.6.3. Interaction modes

The extension should compare three modes. Direct prompting supplies the issue statement and bounded context without tool use. Agentless staged reasoning decomposes the task into localization, patch generation, and validation phases without open-ended autonomous action. Tool-augmented execution grants declared read, search, edit, shell, and test tools under a fixed action budget. Figure 2 summarizes the proposed workflow.

2.6.4. Process telemetry

Process telemetry should be fixed before the main run. Localization quality should be measured as file-level precision, recall, and F1 against the files

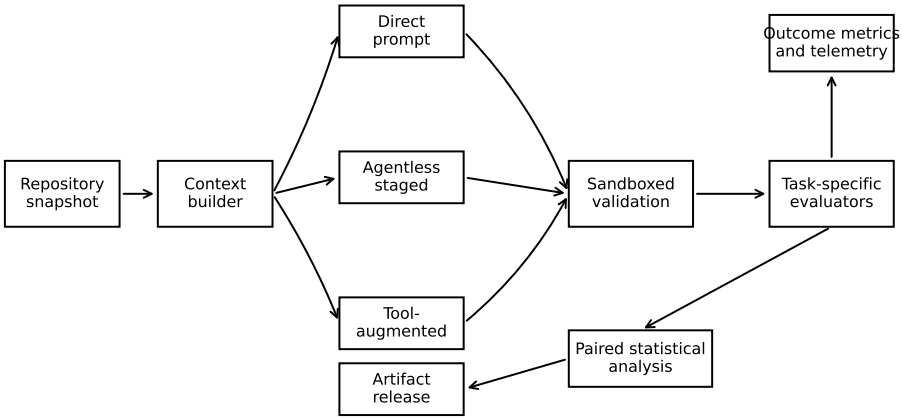


Figure 2: Repository-level evaluation protocol for the proposed follow-up study.

changed by the accepted human patch or benchmark gold patch. Retrieval precision should be computed as the proportion of retrieved files or chunks that overlap oracle files or symbols. A validation loop should count as successful only when the system observes a failing test or diagnostic, changes its patch, and either improves the evaluator state or reaches a passing state. These variables can then be modeled as predictors of final success.

2.6.5. Human review

Human review should use blind expert raters. An expert reviewer should be a senior PhD student, postdoctoral researcher, or software engineer with at least three years of experience in the relevant language or repository ecosystem. The review sample should cover 10% to 15% of instances, stratified by benchmark, model, mode, and automated outcome. The rubric should score correctness, maintainability, minimality, explanation usefulness, and risk awareness. Krippendorff's alpha should be reported, with $\alpha \geq 0.67$ for exploratory claims and $\alpha \geq 0.80$ for confirmatory claims [22].

2.6.6. Statistical analysis

The full-scale study should use paired inference by instance. For binary outcomes such as success or secure success, McNemar tests are appropriate, with mid-p variants preferred when the number of discordant pairs is small or moderate [20]. For paired continuous outcomes such as time, token cost,

patch size, or reviewer score, Wilcoxon signed-rank tests are appropriate. For comparisons across more than two systems and multiple datasets, Friedman tests with corrected post hoc comparisons remain standard practice [21]. Figure 3 gives a compact follow-up timeline.

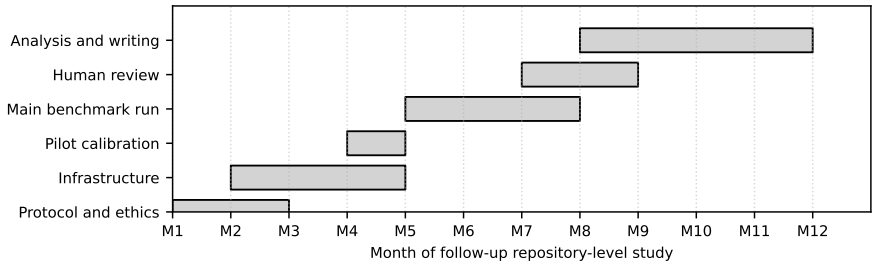


Figure 3: Clean timeline for the proposed repository-level follow-up study.

2.7. SWE-bench Lite execution

The repository-level execution uses 50 instances from the `princeton-nlp/SWE-bench_Lite` test split. The 50-instance slice is treated as an executed diagnostic study rather than a leaderboard-scale claim: it is large enough to expose patch-generation and harness-execution failure modes, while keeping the Docker evaluation and artifact inspection tractable for a first replication package. Two prompting modes are compared on three lightweight backend configurations. The first backend is `gpt-4o-mini`, selected as a low-cost accessible API baseline for testing the pipeline rather than as a state-of-the-art repair model. The official model page lists a 128,000-token context window, text output support, and token prices of US$0.15 per one million input tokens and US$0.60 per one million output tokens [30]. The second backend is `Qwen/Qwen2.5-Coder-7B-Instruct`, selected as a small open-weight code model that can be run outside a proprietary API [26]. The third backend is `Qwen/Qwen2.5-Coder-32B-Instruct-AWQ`, selected to test whether a larger open-weight code model changes the direct-versus-staged pattern. The 32B AWQ model card identifies it as a 4-bit AWQ-quantized instruction-tuned Qwen2.5-Coder model with Apache-2.0 licensing, 32.5B parameters, and long-context support [27]. The Qwen technical report describes the family as code-specialized models for generation, reasoning, and repair [28].

Before each model call, the script clones the target repository at the task's base commit and constructs a bounded repository context. Text files

are scored using overlap between problem-statement tokens and file-path or file-content tokens; the context builder keeps at most 18 files, truncates each file at 12,000 characters, and caps the combined context at 90,000 characters. The direct mode sends one request per instance with the problem statement and this retrieved context. The agentless mode sends two requests per instance: first, a localization prompt asks for strict JSON containing one to five candidate file paths with confidence scores and short reasons; second, a patch-generation prompt receives the problem statement, the localized file list, and a rebuilt context in which localized files are preferred. Both prompts require a unified diff and prohibit prose outside the patch block. The exact prompt files are included in the replication bundle. For every backend and mode, all 50 submitted instances produced non-empty `diff --git` patches.

The generated predictions are evaluated with the official Docker SWE-bench harness. A *completed* instance means that the patch applied and the benchmark tests executed, even if the tests still failed. An *error* instance means that the patch could not be evaluated because of a patch-application or path problem. The paired significance test uses McNemar's exact test, since the two modes are evaluated on the same task instances [20]. Token cost is estimated from logged input and output tokens for the OpenAI run. The Qwen 7B generation job used Hugging Face Jobs with the `a10g-large` hardware flavor. The Qwen 32B AWQ generation job used `l40sx1`. Hugging Face Jobs pricing lists `a10g-large` at US$1.50/hour and `l40sx1` at US$1.80/hour, billed by running time [29]. SWE-bench Lite itself contains 300 test tasks and was designed as a faster, lower-cost subset of SWE-bench [23, 24].

2.8. Larger-run extension protocol

After the 50-instance study, a larger resumable extension was run to test whether the same infrastructure could scale toward the full SWE-bench Lite test split. The extension used the same prompt templates, context builder, repository checkout logic, and direct-versus-agentless staged modes as the 50-instance run. For `gpt-4o-mini` and Qwen 32B AWQ, the target was all 300 SWE-bench Lite test instances. For Qwen 7B, the target was the first 200 instances after lexicographic sorting by `instance_id`. The extension scripts treat the earlier 50-instance predictions as immutable seed data: before generation, each script checks whether an `instance_id` already appears in the old, new, or merged output file, and it skips that instance rather than regenerating it.

Every generated line is normalized to the SWE-bench prediction schema {`instance_id`, `model_patch`, `model_name_or_path`}. If a model output lacks a `diff --git` header, the script writes a minimal valid empty-diff patch rather than leaving the patch field empty. Sidecar JSONL logs record `instance_id`, mode, input tokens, output tokens, elapsed seconds, patch non-emptiness, patch-header status, and any recoverable error. At the end of each script, old and new predictions are merged, deduplicated by `instance_id`, and sorted. The merged predictions are then repaired locally, invalid or empty patches are retained as submitted failures, and repaired non-empty patches are evaluated with the Docker SWE-bench harness using one worker to keep local disk use bounded.

3. Results and Discussion

3.1. Summarization results

Table 1 reports the summarization results. The rule-based baseline has the strongest scores across all three metrics. That outcome is not surprising. The references are short, and the rule system was written around the six function names. The zero-shot language-model baseline gives reasonable paraphrases, but it does not reuse the reference wording as closely. The name-only baseline performs poorly once brevity penalty and multi-token matching are considered.

Table 1: Summarization metric comparison

System	BLEU-1	ROUGE-L	METEOR
Name-only baseline	0.041	0.309	0.130
Rule-based baseline	0.980	0.990	0.981
Zero-shot language-model baseline	0.535	0.565	0.519

Note: METEOR was computed with exact and Porter-stem matches only; synonym matching was disabled in the offline environment.

Figure 4 shows the same comparison. The point of the table is not that a rule-based method is a better summarizer in general. The point is narrower and more useful: with six predictable function names, a deterministic rule system can look excellent under surface-overlap metrics. A broader evaluation is needed before such a result can be treated as engineering evidence.

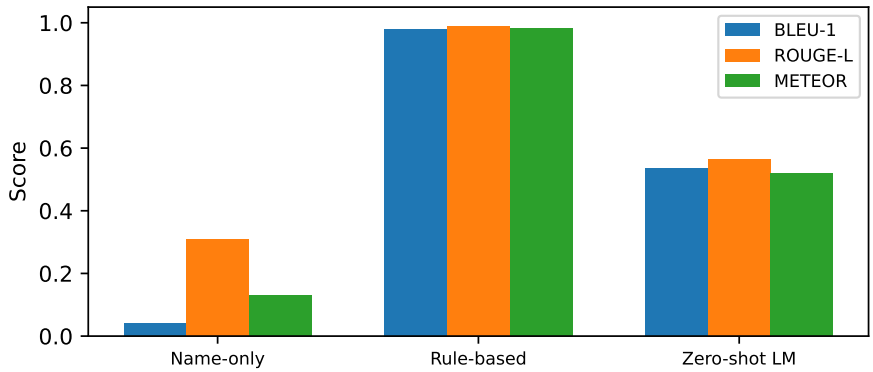


Figure 4: BLEU-1, ROUGE-L, and METEOR results for three summarization systems.

3.2. Bug-detection results

Table 2 reports the bug-detection results. The heuristic obtains 1.00 accuracy on the six-function pilot set, but that number is not the important result. The important result appears on the external stress test. The same detector identifies none of the thirty external bug records. Since it also leaves the paired fixed records clean, specificity is 1.00, recall is 0.00, F1 is 0.00, and overall accuracy is 0.50.

Table 2: Bug-detection results

Dataset	Instances	Accuracy	Recall	F1
Six-function pilot	6	1.00	1.00	1.00
Tests4Py stress set	60	0.50	0.00	0.00

Figure 5 shows the external-stress confusion matrix. The result changes the interpretation of the pilot. The heuristic did not merely perform worse outside the handcrafted examples; it failed to identify any external bug record. That failure is the central empirical finding of the bug-detection part of the study.

3.3. SWE-bench Lite repository results

Table 3 contextualizes the SWE-bench Lite runs before the per-backend results are reported. The comparison is intentionally contextual rather than

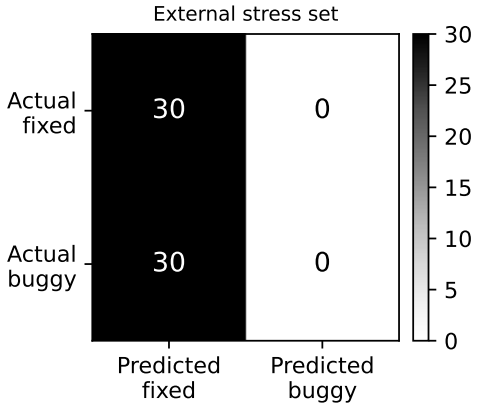


Figure 5: Confusion matrix for the Tests4Py external stress test.

inferential: the public entries use different models, system scaffolds, attempt budgets, and full 300-task Lite submissions, while this paper reports low-cost diagnostic configurations. The table anchors scale. Public full-split systems range from the high 20s to about 60% resolved in the selected rows, whereas the strongest larger-run configuration in this paper resolves 8/300 tasks and the strongest 50-instance diagnostic configuration resolves 4/50 tasks. The present results should therefore be read as low-cost diagnostic baselines, not leaderboard-level systems.

Table 4 reports the 50-instance SWE-bench Lite results for all three backend configurations. With `gpt-4o-mini`, direct prompting submitted 50 patches, completed 23 test executions, produced 27 patch-level errors, and resolved no task. Agentless staged prompting submitted 50 patches, completed 29 test executions, produced 21 patch-level errors, and resolved one task, `django__django-16139`. The paired table contains 49 both-failed tasks, one agentless-only success, no direct-only success, and no both-pass task; McNemar's exact test gives $p = 1.0$.

The Qwen 7B run gives a different but related result. Direct prompting and agentless staged prompting both resolved one task, and both resolved the same instance, `sympy__sympy-14774`. The paired table contains 49 both-failed tasks, no direct-only success, no agentless-only success, and one both-pass task; McNemar's exact test is again $p = 1.0$. Agentless nevertheless improves executability for Qwen 7B: completed harness executions increase from 29 to 33 and patch-application errors fall from 21 to 17.

Table 3: Selected SWE-bench Lite leaderboard context

System or configuration	Scope	Resolved	Role
ExpeRepair-v1.0 + Claude 4 Sonnet	Public 300, 2+ tries	181/300 (60.33%)	Top selected
SWE-agent + Claude 4 Sonnet	Public 300, one try	170/300 (56.67%)	SWE-agent
OpenHands + CodeAct v2.1	Public 300	125/300 (41.67%)	Tool agent
Agentless-1.5 + Claude 3.5 Sonnet	Public 300	122/300 (40.67%)	Staged baseline
Agentless + GPT-4o	Public 300	82/300 (27.33%)	GPT-4o staged
This paper: Qwen 32B AWQ direct	This paper, 300 tasks	8/300 (2.67%)	Larger here
This paper: Qwen 32B AWQ agentless	This paper, 300 tasks	6/300 (2.00%)	Larger staged
This paper: Qwen 32B AWQ agentless	This paper, 50 tasks	4/50 (8.00%)	Best here
This paper: gpt-4o-mini agentless	This paper, 50 tasks	1/50 (2.00%)	Low-cost here

Note: Public rows are selected from the official SWE-bench Lite leaderboard and Lite page [24, 23]. They are not exhaustive and are included to contextualize the scale of this paper's diagnostic and extension results, not to rank these lightweight configurations against public submissions.

The Qwen 32B AWQ run produces the strongest directional result. Direct prompting resolves one task, `scikit-learn__scikit-learn-13439`. Agentless staged prompting resolves four tasks: the same `scikit-learn__scikit-learn-13439` instance plus three agentless-only successes, `django__django-13315`, `django__django-13658`, and `pytest-dev__pytest-11143`. The paired table contains 46 both-failed tasks, no direct-only success, three agentless-only successes, and one both-pass task. McNemar's exact test gives $p = 0.25$, so the result is exploratory rather than statistically significant. This 8% agentless result is the paper's clearest positive signal: it remains a small 50-instance diagnostic run, but it is the only configuration that moves beyond the 2% level observed in the other lightweight backends.

Table 4: SWE-bench Lite execution summary

Backend	Mode	Submitted	Completed	Errors	Resolved	Rate
OpenAI gpt-4o-mini	Direct	50	23	27	0	0.0%
OpenAI gpt-4o-mini	Agentless	50	29	21	1	2.0%
Qwen 7B	Direct	50	29	21	1	2.0%
Qwen 7B	Agentless	50	33	17	1	2.0%
Qwen 32B AWQ	Direct	50	22	28	1	2.0%
Qwen 32B AWQ	Agentless	50	23	27	4	8.0%

Table 5 and Figure 6 break down the outcomes. Across all three backend configurations, staged prompting mainly improves whether patches reach executable form. For OpenAI, malformed patches fall from 18 to 10 and total patch-application errors fall from 27 to 21. For Qwen 7B, official patch-application errors fall from 21 to 17; hunk failures decrease from 11 to 9, malformed patches remain at 8, and file-not-found errors disappear. For Qwen 32B AWQ, patch-application errors fall slightly from 28 to 27 and completed evaluations rise from 22 to 23, while resolved tasks increase from 1 to 4. The 32B direct run has more patch-application errors than the 7B direct run, which is counterintuitive if model size is treated as a simple monotonic improvement. The logs explain the difference as an error-mix shift rather than a pure quality drop: the 32B direct run has more hunk/context failures and introduces a reversed-or-previously-applied category that does not appear in the 7B run. Because patch length was not measured, this paper does not claim that the 32B patches were longer; the observed point is narrower, namely that the larger model generated edits that more often depended on

fragile context or duplicated/reversed existing changes. In the staged 32B run, hunk failures fall from 16 to 12, but malformed patches rise from 6 to 9. Most completed patches still fail the benchmark tests, so the dominant problem is repository-grounded patch validity rather than refusal or empty generation.

Table 5: SWE-bench Lite failure taxonomy

Outcome category	OAI-D	OAI-A	Q7B-D	Q7B-A	Q32-D	Q32-A
Patch applied but tests unresolved	23	28	28	32	21	19
Malformed patch	18	10	8	8	6	9
Hunk/context failure	7	7	11	9	16	12
Reversed/previously applied	0	0	0	0	3	3
Truncated patch	1	3	0	0	0	1
Wrong path/file not found	1	1	2	0	3	2
Resolved	0	1	1	1	1	4

Note: D = direct prompting; A = agentless staged prompting; Q7B = Qwen2.5-Coder-7B-Instruct; Q32 = Qwen2.5-Coder-32B-Instruct-AWQ.

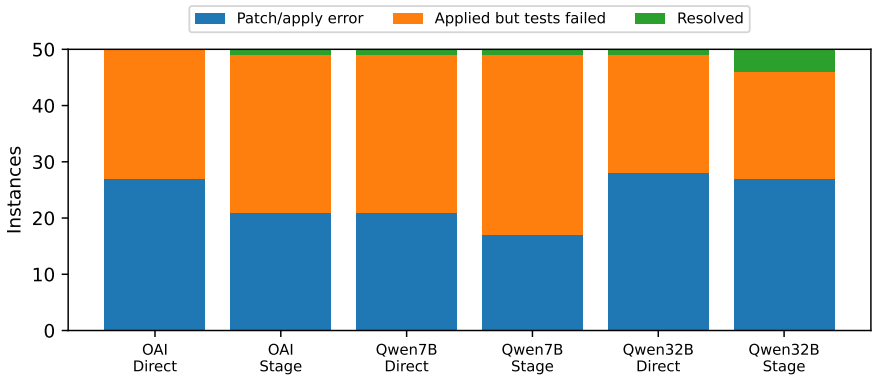


Figure 6: Completed evaluations, patch-application errors, and resolved tasks across the three SWE-bench Lite backend configurations.

The reversed-or-previously-applied errors are also useful qualitatively. They suggest that a repair model can sometimes reconstruct a change that is already present in the base context or emit a patch whose direction is inconsistent with the benchmark state. This is different from a malformed diff: the patch can be syntactically recognizable but semantically misaligned with the repository snapshot.

Table 6 reports cost and runtime. The OpenAI direct mode used 1,033,803 input tokens and 9,311 output tokens across 50 API calls, for an estimated cost of US$0.16. The OpenAI agentless mode used 2,066,916 input tokens and 19,149 output tokens across 100 API calls, for an estimated cost of US$0.32. The total estimated OpenAI API cost was US$0.48. The Qwen 7B generation job ran on Hugging Face Jobs with an `a10g-large` instance for 1,823 running seconds and 1,828 total seconds; at US$1.50 per hour, the estimated generation cost is about US$0.76. The Qwen 32B AWQ generation job ran on `l40sx1` for 1,671 running seconds and 1,745 total seconds; at US$1.80 per hour, the estimated total-seconds cost is about US$0.87 and the running-time-only cost is about US$0.84. For both Hugging Face runs, direct and agentless generations were batched into the same job, so per-mode runtime is not separately available. These figures exclude the local Docker harness cost.

Table 6: SWE-bench Lite generation cost and runtime

Backend	Mode/job	Calls	Runtime	Cost
OpenAI gpt-4o-mini	Direct	50	–	US$0.16
OpenAI gpt-4o-mini	Agentless	100	–	US$0.32
Qwen 7B/HF	Combined job	–	1,828 s	US$0.76
Qwen 32B AWQ/HF	Combined job	–	1,745 s	US$0.87

3.4. Larger-run evaluated extension

Table 7 reports the larger repaired-and-evaluated extension. The `gpt-4o-mini` direct and agentless files reached the full 300-instance target after merging the earlier 50 predictions with 250 newly generated predictions per mode. The Qwen 32B AWQ files also reached the full 300-instance target after the Hugging Face Jobs run completed on an `l40sx1` worker. The Qwen 7B extension uses the first 200 SWE-bench Lite test instances under lexicographic `instance_id` order, matching the pre-registered compute-bounded protocol. Every submitted patch was passed through the same local repair step before Docker evaluation; empty or still-invalid patches were retained as submitted failures rather than dropped.

The generation-cost pattern remains small relative to typical manual repair effort but nonzero. For the 250-new-instance OpenAI extension, direct

generation used 5,127,606 input tokens and 48,459 output tokens for an estimated US$0.798, while agentless generation used 10,253,596 input tokens and 100,551 output tokens for US$1.598. Including the earlier 50-instance slice, the `gpt-4o-mini` generation total is approximately US$2.88. The Qwen 32B AWQ extension job ran for 7,953 seconds wall-clock on `l40sx1`, about US$3.98 at US$1.80/hour; with the earlier 50-instance job, the generation total is about US$4.85. The Qwen 7B extension job ran for 6,323 seconds wall-clock on `a10g-large`, about US$2.63 at US$1.50/hour; with the earlier 50-instance job, the generation total is about US$3.40. These estimates exclude local Docker evaluation cost.

Table 7: Larger repaired-and-evaluated SWE-bench Lite extension

Backend	Mode	Target	Completed	Empty/invalid	Resolved	Rate
OpenAI gpt-4o-mini	Direct	300	151	149	3	1.00%
OpenAI gpt-4o-mini	Agentless	300	157	143	1	0.33%
Qwen 32B AWQ	Direct	300	155	145	8	2.67%
Qwen 32B AWQ	Agentless	300	156	144	6	2.00%
Qwen 7B	Direct	200	91	109	2	1.00%
Qwen 7B	Agentless	200	114	86	2	1.00%

Note: "Completed" means the repaired patch applied and the official SWE-bench Docker harness executed the task. Empty or still-invalid patches are counted in the target denominator as submitted failures. Qwen 7B uses the first 200 test instances under lexicographic `instance_id` order.

Table 8: Paired outcomes in the larger evaluated extension

Backend	n	Both fail	Direct only	Agentless only	Both pass	McNemar p
OpenAI gpt-4o-mini	300	296	3	1	0	0.625
Qwen 32B AWQ	300	289	5	3	3	0.727
Qwen 7B	200	197	1	1	1	1.000

3.5. Interpretation

RQ1 and RQ2 show that metric choice changes how the summarization pilot should be read. BLEU-1 alone was too thin a view of the outputs, because it could not separate brevity effects from sequence overlap or alignment quality. RQ4 is answered by the combined pattern across the pilot and repository layers: methods that look usable under narrow surface checks can fail once external validation is applied. ROUGE-L and METEOR do not

solve semantic evaluation, but they give a less fragile picture than unigram overlap alone. The language-model row should also be kept in its proper place. It is useful as a within-pilot comparison, but it is not a reproducible API benchmark because it was produced through a conversational interface rather than a pinned provider checkpoint with stored request logs.

RQ3 gives the sharper lesson. The initial detector was good at detecting the two bugs it was effectively designed around. It did not transfer to the external records. This is exactly the kind of failure that a small pilot can hide. It also explains why more structured bug-reasoning work, such as the data-flow-constrained approach of Wang et al., places so much emphasis on program evidence rather than ad hoc textual cues [16].

RQ5 adds the repository-level result. Across three backend configurations, staged prompting improves patch executability more consistently than it improves final task resolution. In the OpenAI run, staging produces one agentless-only success and six more completed evaluations. In the Qwen 7B run, staging does not add a unique success, but it reduces patch-application errors and increases completed evaluations by four. In the Qwen 32B AWQ run, staging produces three agentless-only successes and raises the resolve rate from 2% to 8%, but the paired exact test remains non-significant at $p = 0.25$. The reliable conclusion is therefore diagnostic rather than comparative: the bottleneck is not refusal or empty output, but patch validity after repository application and test execution.

RQ6 is answered by the larger repaired-and-evaluated extension in Tables 7 and 8. The larger runs do not preserve the apparent 50-instance Qwen 32B advantage for staged prompting. Instead, `gpt-4o-mini` resolves 3/300 tasks with direct prompting and 1/300 with staged prompting, Qwen 32B AWQ resolves 8/300 with direct prompting and 6/300 with staged prompting, and Qwen 7B resolves 2/200 in both modes. The paired exact tests are again descriptive rather than confirmatory. Staging still improves the number of patches that reach completed harness execution for `gpt-4o-mini` and Qwen 7B, but those executability gains do not translate reliably into more resolved tasks. PDQ1–PDQ4 therefore remain design questions for future work with stronger repository context, tool use, validation loops, and process telemetry.

A practical implication follows for teams evaluating code-agent deployments in software-maintenance workflows. Low-cost staged prompting can be a useful gatekeeping strategy when the goal is to produce patches that at least reach executable form, but the runs here do not justify autonomous merging or reduced review. Teams should track patch-application errors, test

execution, and cost per attempted repair alongside final resolve rate. The extension also shows a process lesson: prediction generation, patch repair, benchmark execution, and paired statistical comparison should be logged as separate stages, because each stage can change the interpretation of a code-agent run.

3.6. Responsible deployment

Language-model tools for software engineering should be run with bounded permissions and auditable validation. Network access should be disabled by default in controlled experiments unless the condition requires it and the access is logged. Secrets, credentials, and private repositories should not be sent to third-party systems without consent and clear data controls. Security-sensitive patches should be evaluated separately from ordinary functional correctness, because a patch can pass a local test suite and still introduce a vulnerability [11]. These points belong in the evaluation design, not only in a deployment checklist.

3.7. Data and artifact availability

A replication bundle is prepared as supplementary material for this submission. It includes the metric script, summary outputs, Tests4Py stress-test record list, plots, LaTeX source, generated figures, prompt templates, final prediction JSONL files, patch-repair summaries, comparison JSON files, and SWE-bench Lite Docker harness summaries for the OpenAI, Qwen 7B, and Qwen 32B AWQ runs. The larger-run extension artifacts include the merged `gpt-4o-mini` and Qwen 32B AWQ 300-instance prediction files, the Qwen 7B 200-instance prediction files, sidecar generation logs, repair outputs, and local harness reports. The complete per-instance prediction and harness logs are not printed in the manuscript PDF, but they are retained in the replication bundle for patch-level inspection and archival deposit.

3.8. Internal validity

The rule-based summarizer was written for the six function names in the pilot. Its strong metric values therefore do not establish general summarization ability. The comparison baselines make the result more interpretable, but they do not remove the small-sample limitation. The bug detector has the same issue on the original two defects. The Tests4Py-derived stress test partially addresses that concern by showing non-transfer. However, because

the stress test uses normalized benchmark metadata instead of full checked-out source states, the finding should be read carefully: the toy patterns do not transfer even to external benchmark descriptors, but this is not yet a full source-level bug-localization experiment. The SWE-bench Lite experiments are stronger because they use the official Docker harness. The initial 50-instance slice is still small, while the larger extension improves scale by evaluating all 300 instances for `gpt-4o-mini` and Qwen 32B AWQ and the first 200 lexicographic instances for Qwen 7B. Empty or invalid patches are retained as submitted failures rather than being dropped, and local Docker harness cost is excluded from the generation-cost tables. The OpenAI direct run in the initial 50-instance slice had a disk/Docker interruption before completion; this affects timing comparability for that mode, but it should not affect the reported resolve counts because the prediction JSONL files were generated before harness execution and the resumed official harness run evaluated the submitted patches.

3.9. Construct validity

BLEU-1, ROUGE-L, and METEOR measure overlap between a generated summary and a reference summary. They improve the earlier BLEU-only setup, but they still do not measure whether a developer would find the summary useful, whether the summary captures the right risk, or whether it is semantically complete. The METEOR implementation also omits synonym matching because WordNet was unavailable offline, so the scores are exact-and-stem METEOR values rather than full-resource METEOR values. Bug-detection accuracy, recall, and F1 are also limited constructs: they do not measure repair quality, causal explanation, security impact, or regression preservation. In the SWE-bench Lite runs, completed test execution is not the same as successful repair; it only means that the patch applied and the harness could run the tests. Patch repair is also a construct choice: it tests whether generated diffs can be normalized into an evaluable artifact, while empty or still-invalid patches are counted as failures. Finally, the zero-shot summarization baseline is illustrative rather than a pinned vendor result, while the repository runs use one named API model and two open-weight Qwen deployments under a minimal context strategy. The 32B Qwen run uses an AWQ-quantized model, so it should be interpreted as that deployment rather than as a full-precision 32B result.

3.10. External validity

The summarization set contains only six small Python functions. Results may not carry over to large repositories, typed languages, frontend systems, distributed services, security-critical code, or mature open-source projects. The Tests4Py stress set is a stronger test than handcrafted bugs, but it is still a small slice of the Python bug space. The SWE-bench Lite runs improve realism, yet they remain experiments on one benchmark split, three lightweight backend configurations, and two prompting modes. They should not be read as a general ranking of direct versus staged prompting or as a leaderboard submission. The larger extension reduces the small-sample concern for `gpt-4o-mini` and Qwen 32B AWQ, but Qwen 7B is evaluated on a deterministic 200-instance subset rather than the full 300-instance split. The zero-shot language-model summarization baseline should also not be compared with published API leaderboards, since it was produced through a ChatGPT model-picker setting rather than a logged API run.

3.11. Conclusion validity

The pilot results are descriptive. The summarization set is too small for inferential statistics, and the external bug stress test intentionally evaluates a narrow heuristic. The initial 50-instance SWE-bench Lite comparisons are paired by instance, but the OpenAI run has only one discordant success, the Qwen 7B run has no discordant successes, and the Qwen 32B AWQ run has three discordant agentless-only successes; McNemar's exact tests are $p = 1.0$, $p = 1.0$, and $p = 0.25$, respectively. The larger extension is also paired by instance and still has few successes, so the exact tests remain descriptive rather than confirmatory. The safest conclusion is therefore not that staged prompting is better in general. The conclusion is narrower: small curated pilots can mislead, and the repository-level runs suggest that patch executability and patch validity should be treated as primary outcomes alongside resolve rate.

4. Conclusion

This paper revises a pilot evaluation of language-model use in software engineering and adds a first repository-level execution. The revision adds ROUGE-L, METEOR, comparison baselines, and an external bug-detection stress test. The rule-based summarizer scores best on six curated Python functions, while the zero-shot language-model baseline behaves differently by paraphrasing more and matching less. The original bug heuristic fails on

the Tests4Py-derived external stress set, detecting none of thirty real bug records.

The SWE-bench Lite runs move the paper beyond a proposal while keeping the claim modest. With gpt-4o-mini on 50 instances, direct prompting resolves 0 tasks and agentless staged prompting resolves 1. With Qwen2.5-Coder-7B-Instruct, both modes resolve the same single task. With Qwen2.5-Coder-32B-Instruct-AWQ, direct prompting resolves 1 task and agentless staged prompting resolves 4, but the paired result is still exploratory ($p = 0.25$). Across the three initial repository configurations, the staged protocol more consistently improves executability than final resolution. The larger extension strengthens the empirical base while narrowing the conclusion: on the 300-instance `gpt-4o-mini` run, direct prompting resolves 3 tasks and staged prompting resolves 1; on the 300-instance Qwen 32B AWQ run, direct prompting resolves 8 and staged prompting resolves 6; on the 200-instance Qwen 7B subset, both modes resolve 2. Thus, the larger evidence does not support a general staged-prompting advantage in final resolution. The useful finding is more diagnostic: lightweight code-agent workflows can be run cheaply and reproducibly at larger scale, but patch validity remains the bottleneck between plausible generated text and repair success. For practitioners, the immediate lesson is to evaluate code agents as software-maintenance tools, with explicit checks for patch validity, test execution, cost, checkpoint recovery, and artifact provenance before any deployment decision. Future work should add stronger repository context, validation loops, and stronger open- and closed-model baselines before making broader claims about engineering utility.

Declarations

CRediT authorship contribution statement: Rishabh Singh: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft, Writing - review and editing, Project administration.
Funding: This research received no external funding.
Competing interests: The author declares no competing financial or personal interests that could have influenced the work reported in this paper.
Data and code availability: The replication bundle is submitted as supplementary material and contains the metric scripts, prompt templates, summarized outputs, generated figures, prediction JSONL files, repair summaries,

comparison JSON files, sidecar generation logs, and Docker harness summaries for the Tests4Py and SWE-bench Lite runs. Complete per-instance prediction and harness logs are included in the bundle for patch-level inspection and archival deposit.

Ethics statement: The study uses public benchmark tasks and does not involve human subjects, private user data, or proprietary repositories.

Declaration of generative AI and AI-assisted technologies in manuscript preparation: During the preparation of this work, the author used OpenAI ChatGPT to assist with language editing, organization, LaTeX formatting, and code-support tasks. The author reviewed, verified, and edited the content and takes full responsibility for the manuscript, experiments, and conclusions.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021, doi: 10.48550/arXiv.2107.03374.

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," arXiv:2108.07732, 2021, doi: 10.48550/arXiv.2108.07732.

[3] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, S. W.-t. Yih, D. Fried, S. Wang, and T. Yu, "DS-1000: A natural and reliable benchmark for data science code generation," arXiv:2211.11501, 2022, doi: 10.48550/arXiv.2211.11501.

[4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," arXiv:2305.01210, 2023, doi: 10.48550/arXiv.2305.01210.

[5] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking repository-level code auto-completion systems," arXiv:2306.03091, 2023, doi: 10.48550/arXiv.2306.03091.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve

real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, doi: 10.48550/arXiv.2405.15793.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024, doi: 10.48550/arXiv.2407.01489.

[9] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "SWE-bench goes live," arXiv:2505.23419, 2025, doi: 10.48550/arXiv.2505.23419.

[10] W. Li, X. Zhang, Z. Guo, S. Mao, W. Luo, G. Peng, Y. Huang, H. Wang, and S. Li, "FEA-Bench: A benchmark for evaluating repository-level code generation for feature implementation," arXiv:2503.06680, 2025, doi: 10.48550/arXiv.2503.06680.

[11] C. Shen, C. Dilgren, P. Chiniya, L. Griffith, Y. Ding, and Y. Chen, "SecRepoBench: Benchmarking code agents for secure code completion in real-world repositories," arXiv:2504.21205, 2025, doi: 10.48550/arXiv.2504.21205.

[12] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv:2302.06590, 2023, doi: 10.48550/arXiv.2302.06590.

[13] E. Paradis, K. Grey, Q. Madison, D. Nam, A. Macvean, V. Meimand, N. Zhang, B. Ferrari-Church, and S. Chandra, "How much does AI impact development speed? An enterprise-based randomized controlled trial," arXiv:2410.12944, 2024, doi: 10.48550/arXiv.2410.12944.

[14] K. Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz, "The effects of generative AI on high-skilled work: Evidence from three field

experiments with software developers," *Management Science*, published online Feb. 27, 2026, doi: 10.1287/mnsc.2025.00535.

[15] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity," arXiv:2507.09089, 2025, doi: 10.48550/arXiv.2507.09089.

[16] C. Wang, W. Zhang, Z. Su, X. Xu, and X. Zhang, "Sanitizing large language models in bug detection with data-flow," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 3790–3805, 2024, doi: 10.18653/v1/2024.findings-emnlp.217.

[17] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, Association for Computational Linguistics, pp. 74–81, 2004. [Online]. Available: `https://aclanthology.org/W04-1013/`. [Accessed: May 9, 2026].

[18] A. Lavie and A. Agarwal, "METEOR: An automatic metric for MT evaluation with high levels of correlation with human judgments," in *Proceedings of the Second Workshop on Statistical Machine Translation*, pp. 228–231, 2007, doi: 10.3115/1626355.1626389.

[19] M. Smytzek, M. Eberlein, B. Serce, L. Grunske, and A. Zeller, "Tests4Py: A benchmark for system testing," in *Proc. ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, 2024, doi: 10.1145/3663529.3663798.

[20] M. W. Fagerland, S. Lydersen, and P. Laake, "The McNemar test for binary matched-pairs data: Mid-p and asymptotic are better than exact conditional," *BMC Medical Research Methodology*, vol. 13, no. 91, 2013, doi: 10.1186/1471-2288-13-91.

[21] J. Demsar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. [Online]. Available: `https://www.jmlr.org/papers/v7/demsar06a.html`

[22] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. Thousand Oaks, CA, USA: Sage, 2018.

[23] SWE-bench Team, "SWE-bench Lite," SWE-bench, 2025. [Online]. Available: https://www.swebench.com/lite.html. [Accessed: May 8, 2026].

[24] SWE-bench Team, "SWE-bench leaderboards," SWE-bench, 2026. [Online]. Available: https://www.swebench.com/. [Accessed: May 19, 2026].

[25] OpenAI, "GPT-5.5 in ChatGPT," OpenAI Help Center, 2026. [Online]. Available: https://help.openai.com/en/articles/11909943-gpt-55-in-chatgpt. [Accessed: May 7, 2026].

[26] Qwen, "Qwen2.5-Coder-7B-Instruct," Hugging Face model card, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct. [Accessed: May 8, 2026].

[27] Qwen, "Qwen2.5-Coder-32B-Instruct-AWQ," Hugging Face model card, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct-AWQ. [Accessed: May 9, 2026].

[28] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Dang, A. Yang, R. Men, F. Huang, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024, doi: 10.48550/arXiv.2409.12186.

[29] Hugging Face, "Pricing and billing," Hugging Face Jobs documentation, 2026. [Online]. Available: https://huggingface.co/docs/hub/en/jobs-pricing. [Accessed: May 8, 2026].

[30] OpenAI, "GPT-4o mini model," OpenAI API Documentation, 2026. [Online]. Available: https://platform.openai.com/docs/models/gpt-4o-mini. [Accessed: May 8, 2026].